

Technical Implementation Outline for BKBC ATI Prediction With SomaScan Proteomics

This checklist operationalizes your existing playbook into an executable, leak-resistant codebase designed for a sub-25-hour hackathon build window. [filecite\[turnOfile0\]](#)

The implementation choices below are anchored in: the Boston Kidney Biopsy Cohort ¹ ATI proteomics paper (156 proteins; Supplementary Data 1 contains the exact biomarker list), the TabPFN foundation model paper (best regime: $\leq 10k$ samples and ≤ 500 features), Stabl (reliability-oriented feature selection for high-p omics), and the proteomic aging clock paper's Boruta+SHAP workflow (a proven, practical recipe for proteomics feature selection and LightGBM generalizability). ²

Data loading, cleaning, and EDA

What to check immediately when data drops

[NEEDS-DATA] **Dataset schema and split format**

- Confirm whether you receive:
- a `train.csv` with labels + a `test.csv` without labels, or
- pre-split folds, or
- a single table with a `split` column.
- Confirm label is *binary* (ATI present/absent) vs *ordinal severity* (e.g., none/mild/moderate/severe). The BKBC study models semiquantitative ATI severity and reports strong associations with specific proteins (SPP1, MRC1, TNC among the top). ³

[NEEDS-DATA] **Missingness and numeric hygiene (proteomics often has oddities)**

- Count missing values per column; identify:
- all-missing columns (drop),
- near-all missing columns (likely drop, unless clinically crucial),
- infinite values (replace with NaN),
- extreme outliers (winsorize only for linear models; GBDTs generally tolerate).
- Decide separate preprocessing per model family:
- LightGBM can natively handle NaNs; imputation is optional.
- Elastic net *requires* imputation + scaling inside CV folds.
- TabPFN is designed to handle missing values with minimal preprocessing, but you must reduce feature count to its validated regime (≤ 500 features). ⁴

[NEEDS-DATA] **Class imbalance and "AUROC traps"**

- Compute positive rate; if $< 10\text{--}15\%$, expect:

- higher variance across folds,
- better gains from repeated CV / seed averaging,
- threshold-free metric (AUROC) still can be misleading if leakage exists.
- Record fold-wise class counts (imbalanced folds can create noisy AUC).

[NEEDS-DATA] **Duplicate patients and near duplicates**

- If there is a patient ID: verify uniqueness; check repeated visits/samples.
- If no ID: detect near duplicates by hashing rounded feature rows (or using cosine similarity of standardized proteomics subsets). This is essential to prevent fold contamination, especially in clinical cohorts.

[NEEDS-DATA] **Aptamer-to-protein mapping**

- Your competition table may be:
- gene-symbol columns (e.g., SPP1),
- SOMAmer SeqIds (e.g., SeqId_13113_7),
- UniProt IDs,
- “Target” names.
- SomaScan file specifications and terminology typically include identifiers like target name and UniProt (and may include SeqId-based analyte IDs), but you should not assume a single convention —build robust matching. ⁵
- Note your challenge indicates ANML normalization + log2 normalization were applied; SomaLogic technical notes describe standardization/normalization pipelines (including ANML as an adaptive normalization step). ⁶

[PRE-CODE] **Data/EDA utilities (ready before data access)**

```
# eda_utils.py
from __future__ import annotations

import re
import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import Optional, Dict, List, Tuple

SEQID_RE = re.compile(r"^SeqId_\d+_\d+$", re.IGNORECASE)

@dataclass
class DataSchemaGuess:
    id_cols: List[str]
    target_col: Optional[str]
    protein_cols: List[str]
    clinical_cols: List[str]
    other_cols: List[str]
```

```

def _normalize_token(s: str) -> str:
    # For matching across naming conventions:
    # - lower
    # - remove non-alnum
    return re.sub(r"^[a-z0-9]+", "", str(s).lower())

def guess_schema(df: pd.DataFrame, target_col: Optional[str] = None, id_col:
Optional[str] = None) -> DataSchemaGuess:
    cols = list(df.columns)

    # Target column guess
    if target_col is None:
        for c in ["target", "label", "y", "ati", "ati_label", "ati_severity"]:
            if c in cols:
                target_col = c
                break

    # ID columns guess
    id_cols = []
    if id_col and id_col in cols:
        id_cols = [id_col]
    else:
        for c in ["patient_id", "person_id", "subject_id", "id", "sample_id"]:
            if c in cols:
                id_cols.append(c)

    # Protein column guess:
    # If any SeqId_* exist, treat those as proteins.
    seqid_cols = [c for c in cols if SEQID_RE.match(str(c))]
    if len(seqid_cols) > 0:
        protein_cols = seqid_cols
        other_candidate = [c for c in cols if c not in protein_cols]
    else:
        # fallback heuristic:
        # proteins usually dominate column count; clinical often small and/or
        # known names
        # we'll begin by numeric columns excluding obvious IDs/target, then
        # refine later
        excluded = set(id_cols + ([target_col] if target_col else []))
        numeric_cols = [c for c in cols if c not in excluded and
pd.api.types.is_numeric_dtype(df[c])]
        # If tons of numeric cols, assume those are proteins+clinical.
        protein_cols = numeric_cols
        other_candidate = [c for c in cols if c not in excluded and c not in
numeric_cols]

    # Clinical guess: if SeqId pattern exists, clinical = numeric not SeqId plus

```

```

non-numeric covariates
    excluded = set(id_cols + ([target_col] if target_col else []))
    if len(seqid_cols) > 0:
        clinical_cols = [c for c in cols if c not in excluded and c not in
protein_cols]
        # remove obvious non-feature columns
        clinical_cols = [c for c in clinical_cols if c not in other_candidate]
    else:
        # fallback: treat "small set of known clinical names" as clinical if
present
        clinical_name_hints =
{"age", "sex", "gender", "race", "egfr", "creatinine", "bun", "uacr", "bmi", "sbp", "dbp"}
        clinical_cols = [c for c in cols if _normalize_token(c) in
clinical_name_hints and c not in excluded]

    other_cols = [c for c in cols if c not in excluded and c not in
protein_cols and c not in clinical_cols]
    return DataSchemaGuess(
        id_cols=id_cols,
        target_col=target_col,
        protein_cols=protein_cols,
        clinical_cols=clinical_cols,
        other_cols=other_cols,
    )

def basic_eda(df: pd.DataFrame, target_col: Optional[str] = None, id_col:
Optional[str] = None) -> Dict[str, object]:
    schema = guess_schema(df, target_col=target_col, id_col=id_col)
    out: Dict[str, object] = {}
    out["n_rows"] = int(df.shape[0])
    out["n_cols"] = int(df.shape[1])
    out["schema_guess"] = schema

    # Missingness
    na_frac = df.isna().mean().sort_values(ascending=False)
    out["top_missing"] = na_frac.head(25)

    # Duplicate rows (exact)
    out["n_duplicate_rows_exact"] = int(df.duplicated().sum())

    # Duplicate IDs
    if schema.id_cols:
        for c in schema.id_cols:
            out[f"n_duplicate_{c}"] = int(df[c].duplicated().sum())

    # Target distribution
    if schema.target_col and schema.target_col in df.columns:
        y = df[schema.target_col]

```

```

    out["target_value_counts"] = y.value_counts(dropna=False)
    # binary guess
    if pd.api.types.is_numeric_dtype(y):
        uniq = sorted([v for v in y.dropna().unique()])
        out["target_unique_values"] = uniq[:50]

    return out

def print_eda_summary(eda: Dict[str, object]) -> None:
    print(f"Rows: {eda['n_rows']:,} Cols: {eda['n_cols']:,}")
    schema = eda["schema_guess"]
    print(f"Guessed target: {schema.target_col}")
    print(f"Guessed IDs: {schema.id_cols}")
    print(f"#protein cols guess: {len(schema.protein_cols):,}")
    print(f"#clinical cols guess: {len(schema.clinical_cols):,}")
    print(f"#other cols guess: {len(schema.other_cols):,}")
    print(f"Exact duplicate rows: {eda['n_duplicate_rows_exact']:,}")
    for k,v in eda.items():
        if k.startswith("n_duplicate_") and k != "n_duplicate_rows_exact":
            print(f"{k}: {v:,}")
    if "target_value_counts" in eda:
        print("\nTarget distribution:")
        print(eda["target_value_counts"])
    print("\nTop missingness:")
    print(eda["top_missing"])

```

Common failure modes to avoid (decision rules):

- If you see **perfectly duplicated** rows across “train” and “test”, treat as potential leakage artifact and verify the competition rules (rare but severe).
- If **>1%** of rows are exact duplicates, prioritize group CV and/or deduping strategy before any modeling.
- If the label appears ordinal and you’re unsure how AUROC is computed, immediately build both:
 - a binary mapping (e.g., (moderate|severe)=1) and
 - an ordinal score mapping (0–3) used as a “ranking score” (AUROC is invariant to monotonic transforms).

CV harness design

Intended design principles

Your CV harness should enforce: **every transformation is fit on the training fold and applied to the validation fold**. scikit-learn explicitly calls this out as a common pitfall, and empirical work shows feature selection outside CV can create large positive bias in AUC. ⁷

[NEEDS-DATA] Choose the correct splitter

- If you have patient IDs:
 - use `StratifiedGroupKFold` (preferred) or `GroupKFold` with manual stratification.
- Otherwise:
 - use `StratifiedKFold` for binary,
 - for ordinal: stratify on binned severity classes.
- Use **5 folds** as default; go to **10 folds** only if folds are stable and training is fast.

[NEEDS-DATA] OOF collection

- Always produce:
 - per-model out-of-fold (OOF) predictions for blending/stacking,
 - per-fold fitted models only if needed (avoid memory blow-ups; store seeds + params instead),
 - per-fold feature lists selected inside fold (for stability analysis).

[NEEDS-DATA] Leakage map (must be enforced)

- Must be inside fold:
 - imputation,
 - scaling,
 - feature selection (Boruta/SHAP selection, elastic net selection, any filter selection),
 - target-derived features (avoid completely; no target encoding unless categorical and done properly).

[PRE-CODE] CV harness and OOF engine

```
# cv_harness.py
from __future__ import annotations

import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import Callable, Dict, Optional, Tuple, List, Any

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score

try:
    from sklearn.model_selection import StratifiedGroupKFold
except Exception:
    StratifiedGroupKFold = None # older sklearn

@dataclass
class CVConfig:
    n_splits: int = 5
    shuffle: bool = True
    random_state: int = 42
```

```

group_col: Optional[str] = None # patient id if exists

def make_splitter(cfg: CVConfig, y: np.ndarray, groups: Optional[np.ndarray] =
None):
    if cfg.group_col is not None and groups is None:
        raise ValueError("group_col set but groups not provided.")

    if cfg.group_col is not None:
        if StratifiedGroupKFold is None:
            raise RuntimeError("StratifiedGroupKFold not available in this
sklearn version.")
        return StratifiedGroupKFold(n_splits=cfg.n_splits, shuffle=cfg.shuffle,
random_state=cfg.random_state)
        return StratifiedKFold(n_splits=cfg.n_splits, shuffle=cfg.shuffle,
random_state=cfg.random_state)

@dataclass
class OOFResult:
    oof_pred: np.ndarray
    fold_auc: List[float]
    mean_auc: float
    std_auc: float
    meta: Dict[str, Any]

def run_oof_cv(
    X: pd.DataFrame,
    y: np.ndarray,
    fit_predict_fn: Callable[[pd.DataFrame, np.ndarray, pd.DataFrame, Dict[str,
Any]], np.ndarray],
    cfg: CVConfig,
    groups: Optional[np.ndarray] = None,
    fit_params: Optional[Dict[str, Any]] = None,
) -> OOFResult:
    """
    fit_predict_fn: function that fits on (X_tr, y_tr) and returns proba for
X_va.
    Must include all preprocessing/feature selection INSIDE the function.
    """
    fit_params = fit_params or {}
    splitter = make_splitter(cfg, y=y, groups=groups)

    oof = np.zeros(len(X), dtype=float)
    fold_auc: List[float] = []
    fold_meta: List[Dict[str, Any]] = []

    if cfg.group_col is not None:
        split_iter = splitter.split(X, y, groups=groups)
    else:

```

```

split_iter = splitter.split(X, y)

for fold, (tr_idx, va_idx) in enumerate(split_iter):
    X_tr, y_tr = X.iloc[tr_idx], y[tr_idx]
    X_va, y_va = X.iloc[va_idx], y[va_idx]

    pred_va = fit_predict_fn(X_tr, y_tr, X_va, fit_params)
    oof[va_idx] = pred_va

    auc = roc_auc_score(y_va, pred_va)
    fold_auc.append(float(auc))
    fold_meta.append({"fold": fold, "n_train": len(tr_idx), "n_valid":
len(va_idx), "auc": float(auc)})

    print(f"[fold {fold}] AUC={auc:.5f} n_train={len(tr_idx)}
n_valid={len(va_idx)}")

mean_auc = float(np.mean(fold_auc))
std_auc = float(np.std(fold_auc))
print(f"OOF mean AUC={mean_auc:.5f} ± {std_auc:.5f}")

return OOFResult(
    oof_pred=oof,
    fold_auc=fold_auc,
    mean_auc=mean_auc,
    std_auc=std_auc,
    meta={"fold_meta": fold_meta, "cv_config": cfg},
)

```

Decision rules:

- If fold AUC standard deviation > **0.03** on 5-fold CV, prioritize:
 - stronger regularization,
 - feature subsampling (GBDT),
 - fewer features,
- repeated CV seeds (stability wrapper), before “fancy models.”
- If group IDs exist and you can’t stratify well, prefer **more folds** (e.g., 7–10) to reduce group imbalance per fold, but only if training time permits.

Feature subset ladder with BKBC Supplementary Data 1 proteins

Evidence-based subsets you should predefine

The BKBC ATI paper reports **156 proteins** significantly linked to ATI and provides the full list in **Supplementary Data 1** (170 aptamers mapping to 156 unique proteins). ⁸

[NEEDS-DATA] FS0: clinical-only

- All non-proteomics covariates available at prediction time.
- Purpose: quantify proxy power (eGFR often strong).

[NEEDS-DATA] FS1: clinical + published ATI biomarkers

- Use the exact biomarker list from Supplementary Data 1.
- Must support matching by:
 - gene symbol (e.g., SPP1, MRC1, TNC, HAVCR1),
 - SOMAmer SeqId (e.g., SeqId_13113_7 for osteopontin in that supplement),
 - UniProt IDs,
 - "Target" names.
- This ladder rung is the fastest evidence-backed route to a strong leaderboard baseline.

[NEEDS-DATA] FS2: expanded data-driven set

- Start from FS1, then add top-k features selected **inside CV** via:
 - fold-wise univariate filter (fast),
 - then fold-wise SHAP ranking from a GBDT,
 - or stability selection frequency.

[PRE-CODE] BKBC FS1 exact protein definitions and selector

The code below embeds the **exact 156 gene symbols** and **170 SeqIds** extracted from Supplementary Data 1 of the ATI paper (and provides robust matching to whatever naming convention the competition uses).

9

```
# bkbcs_fs1.py
from __future__ import annotations

import re
from typing import Dict, List, Set, Tuple
import pandas as pd

SEQID_RE = re.compile(r"^SeqId_\d+_\d+$", re.IGNORECASE)

def _norm(s: str) -> str:
    return re.sub(r"^[a-z0-9]+", "", str(s).lower())

# Exact BKBC ATI biomarker list (Supplementary Data 1; 156 proteins, 170
# aptamers)
# Source: Supplementary Data 1 in "Plasma proteomics of acute tubular
# injury" (Nat Commun 2024).
BKBC_ATI_156_GENES: List[str] = ['ACP5', 'ADAMTSL2', 'AFM', 'AGRN', 'AHSG',
'ALDH7A1', 'ANGPT2', 'ANTXR2', 'APOA1', 'APOA4', 'ATOX1', 'B2M', 'BAGE3',
'BAMBI', 'BCEH', 'BICDL1', 'BMPER', 'C1QTNF1', 'C1QTNF3', 'C5', 'C5|C6', 'CA6',
```

'CACNA2D3', 'CALCB', 'CBLN4', 'CCDC126', 'CCN1', 'CD163', 'CD58', 'CDH3',
 'CDH5', 'CEACAM1', 'CHGA', 'CHST7', 'CHSY3', 'CLDN5', 'CLIC5', 'CNN3',
 'COL3A1', 'COL6A1', 'CPB2', 'CTSC', 'CTSD', 'CXCL16', 'DAB2', 'DDAH2', 'DLD',
 'DNMBP', 'DSC2', 'DST', 'ECM1', 'EFEMP1', 'EGFL7', 'ENPP2', 'EPHB4', 'EREG',
 'ESAM', 'F10', 'FABP3', 'FBLN2', 'FCGR2B', 'FGF7', 'FGL1', 'FIBIN', 'FKBP5',
 'FOLR3', 'GAL', 'GDF15', 'GDF2', 'GFI1', 'GOLM1', 'GPX3', 'GZMB', 'HABP2',
 'HAVCR1', 'HBG1', 'HMOX1', 'HOXD13', 'HP', 'HPX', 'ICAM5', 'IFNG', 'IGFBP7',
 'IL15RA', 'IL18', 'IL1R1', 'IL1RN', 'IL2RA', 'IL6', 'IL7R', 'INHBA', 'ITGA11',
 'ITIH4', 'JCHAIN', 'JUP', 'KAZALD1', 'KCNIP2', 'KRT18', 'KRT19', 'KRT7',
 'KRT8', 'KRTCAP2', 'KRTCAP3', 'L1CAM', 'LAMB1', 'LBP', 'LGALS3', 'LILRA5',
 'LILRB2', 'LRRC15', 'LTF', 'LY6E', 'LYVE1', 'MMP7', 'MMP8', 'MRC1', 'MSTN',
 'MUC16', 'MUC1', 'MZB1', 'NCAM1', 'NDRG1', 'NENF', 'NPS', 'NTN4', 'OLFM4',
 'OPTC', 'OSMR', 'PCSK7', 'PDIA3', 'PDIA6', 'PDPN', 'PFN1', 'PLA2G2A', 'PLAT',
 'PLEK', 'PLXNA1', 'PODXL', 'PPIB', 'PRSS8', 'PRTN3', 'PSEN1', 'PTN', 'PTRJ',
 'PVR', 'QSOX1', 'RBP7', 'RELT', 'RGS5', 'RIPK2', 'RNASE1', 'RNASE3', 'RPLP0',
 'RPS10', 'RPS11', 'RPS27', 'SCG2', 'SCGB3A1', 'SCT', 'SEMA3F', 'SERPINA4',
 'SERPINA5', 'SLC25A18', 'SLC7A5', 'SMOC1', 'SOST', 'SPP1', 'STC1', 'STC2',
 'STMN1', 'SULT1A1', 'TACC2', 'TACSTD2', 'TBC1D10A', 'TCL1A', 'TFF3', 'TGFB1',
 'TIMP1', 'TNC', 'TNFRSF11B', 'TNFRSF12A', 'TNFRSF1B', 'TNFSF13B', 'TPPP3',
 'TRAILR4', 'TRIM5', 'TSLP', 'TUBB', 'VCAN', 'WFDC2']

170 aptamer SeqIds from Supplementary Data 1.

BKBC_ATI_170_SEQIDS: List[str] = ['SeqId_1014_19', 'SeqId_10366_17',
 'SeqId_1038_52', 'SeqId_10524_32', 'SeqId_10609_15', 'SeqId_10610_62',
 'SeqId_10632_14', 'SeqId_1078_37', 'SeqId_10846_8', 'SeqId_1097_48',
 'SeqId_11066_72', 'SeqId_1111_18', 'SeqId_11129_61', 'SeqId_11137_32',
 'SeqId_1118_31', 'SeqId_11263_12', 'SeqId_11609_38', 'SeqId_11610_41',
 'SeqId_11790_32', 'SeqId_1185_14', 'SeqId_11923_31', 'SeqId_1203_18',
 'SeqId_1206_25', 'SeqId_12142_20', 'SeqId_1232_59', 'SeqId_1252_44',
 'SeqId_1253_50', 'SeqId_1254_6', 'SeqId_1267_18', 'SeqId_12768_1',
 'SeqId_1289_57', 'SeqId_13113_7', 'SeqId_13335_22', 'SeqId_13376_6',
 'SeqId_13460_18', 'SeqId_13507_23', 'SeqId_13660_76', 'SeqId_1370_11',
 'SeqId_13724_8', 'SeqId_1372_44', 'SeqId_1376_26', 'SeqId_1382_47',
 'SeqId_13935_10', 'SeqId_14051_14', 'SeqId_14105_5', 'SeqId_14107_13',
 'SeqId_1419_34', 'SeqId_1426_44', 'SeqId_14413_94', 'SeqId_1455_22',
 'SeqId_14754_3', 'SeqId_14860_1', 'SeqId_14944_48', 'SeqId_15008_3',
 'SeqId_1504_41', 'SeqId_15067_23', 'SeqId_1507_44', 'SeqId_1509_18',
 'SeqId_1512_64', 'SeqId_1529_22', 'SeqId_15440_10', 'SeqId_15514_26',
 'SeqId_15515_2', 'SeqId_15561_8', 'SeqId_1567_7', 'SeqId_1586_24',
 'SeqId_1602_21', 'SeqId_1608_2', 'SeqId_16172_22', 'SeqId_16322_10',
 'SeqId_16401_52', 'SeqId_16616_10', 'SeqId_1672_30', 'SeqId_16911_7',
 'SeqId_1724_10', 'SeqId_1732_1', 'SeqId_1732_50', 'SeqId_17456_53',
 'SeqId_1761_7', 'SeqId_17645_1', 'SeqId_17756_13', 'SeqId_18031_56',
 'SeqId_18170_5', 'SeqId_1818_6', 'SeqId_18196_8', 'SeqId_1824_16',
 'SeqId_18331_10', 'SeqId_18374_47', 'SeqId_18407_1', 'SeqId_1856_30',
 'SeqId_1897_24', 'SeqId_1937_44', 'SeqId_1954_70', 'SeqId_19649_72',
 'SeqId_2001_28', 'SeqId_2037_33', 'SeqId_21340_52', 'SeqId_2193_52',
 'SeqId_22681_23', 'SeqId_23173_3', 'SeqId_23288_50', 'SeqId_23383_1',

```
'SeqId_23492_17', 'SeqId_23767_1', 'SeqId_24007_50', 'SeqId_24224_1',
'SeqId_2440_2', 'SeqId_24611_36', 'SeqId_24655_5', 'SeqId_2495_68',
'SeqId_2516_123', 'SeqId_2573_20', 'SeqId_25967_34', 'SeqId_2599_25',
'SeqId_2602_2', 'SeqId_2603_12', 'SeqId_261_43', 'SeqId_2610_13',
'SeqId_2621_20', 'SeqId_2637_77', 'SeqId_26496_11', 'SeqId_2799_33',
'SeqId_2866_6', 'SeqId_2890_5', 'SeqId_2910_3', 'SeqId_3022_25',
'SeqId_3098_10', 'SeqId_3117_7', 'SeqId_3232_28', 'SeqId_3327_27',
'SeqId_3449_58', 'SeqId_3710_8', 'SeqId_4073_10', 'SeqId_4155_3',
'SeqId_4204_12', 'SeqId_4322_33', 'SeqId_4336_2', 'SeqId_4488_33',
'SeqId_452_41', 'SeqId_4603_1', 'SeqId_4673_13', 'SeqId_4763_31',
'SeqId_495_7', 'SeqId_496_73', 'SeqId_504_19', 'SeqId_5123_22', 'SeqId_5675_6',
'SeqId_5738_25', 'SeqId_574_2', 'SeqId_586_19', 'SeqId_595_65', 'SeqId_6114_1',
'SeqId_6120_28', 'SeqId_613_56', 'SeqId_6260_14', 'SeqId_6379_62',
'SeqId_650_1', 'SeqId_681_1', 'SeqId_689_27', 'SeqId_7055_13',
'SeqId_7131_207', 'SeqId_7162_11', 'SeqId_7348_12', 'SeqId_759_14',
'SeqId_7674_38', 'SeqId_7780_34', 'SeqId_7787_25', 'SeqId_7792_7',
'SeqId_7799_11', 'SeqId_7802_20', 'SeqId_7818_1', 'SeqId_7938_15',
'SeqId_8036_33', 'SeqId_8239_223', 'SeqId_8339_235', 'SeqId_8459_78',
'SeqId_8563_27', 'SeqId_8642_3', 'SeqId_8766_29', 'SeqId_8835_3',
'SeqId_8983_7', 'SeqId_9137_55', 'SeqId_9170_34', 'SeqId_9356_6',
'SeqId_9926_12']
```

Optional: known markers explicitly highlighted in the paper (sanity check set)

```
KNOWN_ATI_MARKERS = ["SPP1", "MRC1", "TNC", "NTN4", "MZB1", "HAVCR1", "WFDC2",
"GDF15", "SAA1", "SAA2"]
```

```
def build_fs1_token_set() -> Set[str]:
    # Tokens used to match dataset columns against BKBC FS1 list.
    tokens = set()
    for g in BKBC_ATI_156_GENES:
        tokens.add(_norm(g))
    for sid in BKBC_ATI_170_SEQIDS:
        tokens.add(_norm(sid))
    # Many datasets encode target/protein names in column headers; we can also
    match the
    # gene list directly. If organizers provide UniProt or target names, add
    those here.
    return tokens
```

```
def select_fs1_columns(df: pd.DataFrame, clinical_cols: List[str]) ->
Tuple[List[str], Dict[str, List[str]]]:
    """
    Returns:
        fs1_cols: clinical + any columns matching the FS1 token set
        diag: diagnostics about matches
    """
    token_set = build_fs1_token_set()
    matched = []
```

```

unmatched = []
for c in df.columns:
    if c in clinical_cols:
        continue
    if _norm(c) in token_set:
        matched.append(c)
    else:
        unmatched.append(c)

diag = {
    "n_matched": [str(len(matched))],
    "example_matched": matched[:20],
    "example_unmatched": unmatched[:20],
}
fs1_cols = list(dict.fromkeys(clinical_cols + matched))
return fs1_cols, diag

```

Decision rules:

- If FS1 selection matches **<50 columns**, it likely means the dataset uses a different naming convention (e.g., UniProt or human-readable target names). In that case, immediately request/locate the competition's annotation file and extend `build_fs1_token_set()` using that mapping.
- If FS1 selection matches **thousands of columns**, your matching is too permissive; tighten normalization or require exact matches on SeqId/gene.

Feature selection methods implemented strictly inside CV folds

Recommended ordering for a 25-hour hackathon

The proteomic age clock paper provides a very practical pattern: fit LightGBM, then Boruta (shadow features) using SHAP, then recursive elimination using SHAP to compress panels while preserving performance. ¹⁰

Stabl's central point is that L1-based sparsity is unstable under small perturbations in high-p omics; Stabl injects noise and uses a data-driven threshold to yield *reliable* sparse biomarker sets while maintaining predictive performance. ¹¹

Given hackathon constraints, your best engineering trade is: 1) fast fold-wise filter to reduce p, 2) fold-wise SHAP ranking on a GBDT, 3) optional Boruta within a reduced candidate set, 4) stability wrapper across multiple seeds.

[NEEDS-DATA] Candidate pool strategy (critical runtime control)

- Do *not* run Boruta over 6,500 proteins at once.
- Create a candidate pool per fold:
- FS1 proteins + top-k univariate from remaining proteins (k 500–2,000).

- Run SHAP/Boruta only on candidate pool.

[PRE-CODE] Fold-wise selectors: SHAP ranking, Boruta-style, elastic net stability

```
# feature_selectors.py
from __future__ import annotations

import numpy as np
import pandas as pd
from typing import List, Tuple, Dict, Optional
from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

def fast_filter_topk_mutual_info(X_tr: pd.DataFrame, y_tr: np.ndarray, k: int,
                                random_state: int = 42) -> List[str]:
    """
    Very fast filter. Always run INSIDE the fold.
    """
    # Mutual info cannot handle NaNs; impute inside fold only.
    imp = SimpleImputer(strategy="median")
    X_imp = imp.fit_transform(X_tr)
    mi = mutual_info_classif(X_imp, y_tr, random_state=random_state,
                             discrete_features=False)
    mi = np.nan_to_num(mi, nan=0.0, posinf=0.0, neginf=0.0)
    idx = np.argsort(-mi)[:k]
    return list(X_tr.columns[idx])

def train_lightgbm_for_shap(
    X_tr: pd.DataFrame, y_tr: np.ndarray, X_va: pd.DataFrame,
    params: Dict
) -> Tuple[np.ndarray, Optional[np.ndarray]]:
    """
    Fits LightGBM and returns:
    pred_va: validation probabilities
    shap_importance: mean(|SHAP|) per feature if shap is available else None.
    """
    try:
        import lightgbm as lgb
    except Exception as e:
        raise RuntimeError("LightGBM not installed") from e

    # LightGBM handles NaNs; clean infs.
    X_tr = X_tr.replace([np.inf, -np.inf], np.nan)
    X_va = X_va.replace([np.inf, -np.inf], np.nan)
```

```

model = lgb.LGBMClassifier(**params)
model.fit(
    X_tr, y_tr,
    eval_set=[(X_va, y_tr[:len(X_va)] if len(y_tr) == len(X_va) else
y_tr)], # placeholder, overwritten below
    eval_metric="auc",

callbacks=[lgb.early_stopping(stopping_rounds=params.get("early_stopping_rounds",
100), verbose=False)]
)
pred_va = model.predict_proba(X_va)[:, 1]

# Optional SHAP
shap_imp = None
try:
    import shap
    expl = shap.TreeExplainer(model)
    # For speed, subsample rows if huge
    X_sample = X_tr.sample(n=min(2000, len(X_tr)), random_state=0)
    shap_vals = expl.shap_values(X_sample)
    # shap_vals may be list for binary; handle both
    if isinstance(shap_vals, list):
        shap_vals = shap_vals[1]
    shap_imp = np.mean(np.abs(shap_vals), axis=0)
except Exception:
    shap_imp = None

return pred_va, shap_imp

def boruta_shadow_select(
    X_tr: pd.DataFrame, y_tr: np.ndarray,
    base_lgb_params: Dict,
    max_iter: int = 10,
    keep_ratio: float = 1.0,
    random_state: int = 42,
) -> List[str]:
    """
    Boruta-style selection using shadow (permuted) features with LightGBM
    importance.

    This mimics the Boruta+SHAP shadow-feature logic described in the proteomic
    aging clock paper.

    Keep features whose importance exceeds max shadow importance (strict) or a
    quantile (keep_ratio<1).
    """
    try:
        import lightgbm as lgb
    except Exception as e:
        raise RuntimeError("LightGBM not installed") from e

```

```

rng = np.random.default_rng(random_state)
cols = list(X_tr.columns)
keep = np.ones(len(cols), dtype=bool)

for it in range(max_iter):
    cur_cols = [c for c, k in zip(cols, keep) if k]
    if len(cur_cols) <= 5:
        break

    X_cur = X_tr[cur_cols].copy()
    # Create shadow features by permuting each column
    X_shadow = X_cur.apply(lambda s: rng.permutation(s.values))
    X_shadow.columns = [f"shadow_{c}" for c in X_cur.columns]

    X_aug = pd.concat([X_cur, X_shadow], axis=1)

    model = lgb.LGBMClassifier(**base_lgb_params)
    model.fit(X_aug, y_tr)

    # Gain importance
    imp = model.booster_.feature_importance(importance_type="gain")
    feat_names = model.booster_.feature_name()

    imp_map = dict(zip(feat_names, imp))
    shadowimps = np.array([imp_map[f"shadow_{c}"] for c in X_cur.columns])
    thresh = np.max(shadowimps) if keep_ratio >= 1.0 else
float(np.quantile(shadowimps, keep_ratio))

    # Update keep mask
    new_keep = []
    for c in cols:
        if not keep[cols.index(c)]:
            new_keep.append(False)
            continue
        if c not in X_cur.columns:
            new_keep.append(False)
            continue
        new_keep.append(imp_map.get(c, 0.0) > thresh)
    keep = np.array(new_keep, dtype=bool)

    return [c for c, k in zip(cols, keep) if k]

def elasticnet_stability_select(
    X_tr: pd.DataFrame, y_tr: np.ndarray,
    n_boot: int = 50,
    subsample: float = 0.7,
    C: float = 0.2,

```

```

    l1_ratio: float = 0.5,
    min_freq: float = 0.6,
    random_state: int = 42,
) -> List[str]:
    """
    Stability selection wrapper for elastic net logistic regression.
    Motivated by omics instability concerns highlighted by Stabl; this is a
    hackathon-feasible approximation.
    """
    rng = np.random.default_rng(random_state)
    cols = np.array(X_tr.columns)
    counts = np.zeros(len(cols), dtype=int)

    for b in range(n_boot):
        idx = rng.choice(len(X_tr), size=int(subsample * len(X_tr)),
replace=False)
        Xb = X_tr.iloc[idx].replace([np.inf, -np.inf], np.nan)
        yb = y_tr[idx]

        # Impute + scale INSIDE bootstrap
        imp = SimpleImputer(strategy="median")
        sc = StandardScaler(with_mean=True, with_std=True)

        X_imp = imp.fit_transform(Xb)
        X_scaled = sc.fit_transform(X_imp)

        clf = LogisticRegression(
            penalty="elasticnet",
            solver="saga",
            C=C,
            l1_ratio=l1_ratio,
            max_iter=5000,
            n_jobs=-1,
            class_weight="balanced",
            random_state=int(rng.integers(0, 1_000_000))
        )
        clf.fit(X_scaled, yb)
        coef = clf.coef_.ravel()
        sel = np.where(np.abs(coef) > 1e-12)[0]
        counts[sel] += 1

    freq = counts / n_boot
    selected = cols[freq >= min_freq].tolist()
    return selected

```

Decision rules:

- If SHAP computation is too slow, fall back to LightGBM gain importance for ranking, but keep SHAP for final interpretability checks.
- If Boruta selection returns >2,000 features, your candidate pool was too large or data is noisy; revert to SHAP top-k.
- If elastic net stability selects wildly different sets across seeds, reduce feature pool first (e.g., FS1 + top-k filter) before stability selection.

Model implementations with exact hyperparameter starting points

Practical SOTA implication for your regime

Your problem regime ($N \approx 1,500$; $p \approx 6,500$) matches the regime where tree-based models (GBDTs) typically remain dominant on tabular data benchmarks and generally require less tuning than deep tabular models. ¹²

TabPFN is compelling as an ensemble component once you reduce to ≤ 500 features; the TabPFN paper explicitly states its dominant validated regime is datasets up to 10,000 samples and 500 features, and it is designed for minimal preprocessing with missing values/categorical handling. ¹³

TabICL is mainly relevant conceptually (ICL scaling to large datasets); in practice, unless preinstalled/ provided, it's not a hackathon day-one dependency. ¹⁴

[PRE-CODE] Model factories and fold-wise fit/predict functions

```
# models_and_fitfns.py
from __future__ import annotations

import numpy as np
import pandas as pd
from typing import Dict, Any, List, Optional, Tuple

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

def make_lgb_params(seed: int = 42) -> Dict[str, Any]:
    # Strong default for high-p, medium-N; tuned later if needed.
    return dict(
        objective="binary",
        boosting_type="gbdt",
        n_estimators=5000,
        learning_rate=0.03,
        num_leaves=63,
```

```

        min_data_in_leaf=50,
        feature_fraction=0.20,
        bagging_fraction=0.80,
        bagging_freq=1,
        lambda_l1=0.10,
        lambda_l2=5.00,
        max_depth=-1,
        n_jobs=-1,
        random_state=seed,
        # for our harness
        early_stopping_rounds=150,
    )

```

```
def make_xgb_params(seed: int = 42) -> Dict[str, Any]:
```

```

    return dict(
        n_estimators=8000,
        learning_rate=0.03,
        max_depth=4,
        min_child_weight=10.0,
        subsample=0.8,
        colsample_bytree=0.25,
        reg_alpha=0.1,
        reg_lambda=5.0,
        gamma=0.0,
        objective="binary:logistic",
        eval_metric="auc",
        tree_method="hist",
        n_jobs=-1,
        random_state=seed,
    )

```

```
def fit_predict_lightgbm(X_tr: pd.DataFrame, y_tr: np.ndarray, X_va:
pd.DataFrame, params: Dict[str, Any]) -> np.ndarray:
```

```
    import lightgbm as lgb
```

```
    X_tr = X_tr.replace([np.inf, -np.inf], np.nan)
```

```
    X_va = X_va.replace([np.inf, -np.inf], np.nan)
```

```
    # Use a proper validation set for early stopping by splitting inside fold.
```

```
    # Hackathon shortcut: use 10% of X_tr as internal validation.
```

```
    # For strictness you can pass X_va separately; but we want minimal wiring.
```

```
    inner_frac = 0.1
```

```
    n = len(X_tr)
```

```
    n_val = max(1, int(inner_frac * n))
```

```
    X_inner_val = X_tr.iloc[:n_val]
```

```
    y_inner_val = y_tr[:n_val]
```

```
    X_inner_tr = X_tr.iloc[n_val:]
```

```
    y_inner_tr = y_tr[n_val:]

```

```

        model = lgb.LGBMClassifier(**{k:v for k,v in params.items() if k !=
"early_stopping_rounds"})
        model.fit(
            X_inner_tr, y_inner_tr,
            eval_set=[(X_inner_val, y_inner_val)],
            eval_metric="auc",

callbacks=[lgb.early_stopping(stopping_rounds=params.get("early_stopping_rounds",
150), verbose=False)]
        )
        return model.predict_proba(X_va)[:, 1]

def fit_predict_xgboost(X_tr: pd.DataFrame, y_tr: np.ndarray, X_va:
pd.DataFrame, params: Dict[str, Any]) -> np.ndarray:
    import xgboost as xgb

    X_tr = X_tr.replace([np.inf, -np.inf], np.nan)
    X_va = X_va.replace([np.inf, -np.inf], np.nan)

    inner_frac = 0.1
    n = len(X_tr)
    n_val = max(1, int(inner_frac * n))
    X_inner_val = X_tr.iloc[:n_val]
    y_inner_val = y_tr[:n_val]
    X_inner_tr = X_tr.iloc[n_val:]
    y_inner_tr = y_tr[n_val:]

    model = xgb.XGBClassifier(**params)
    model.fit(
        X_inner_tr, y_inner_tr,
        eval_set=[(X_inner_val, y_inner_val)],
        verbose=False,
        early_stopping_rounds=200
    )
    return model.predict_proba(X_va)[:, 1]

def fit_predict_elasticnet(X_tr: pd.DataFrame, y_tr: np.ndarray, X_va:
pd.DataFrame, params: Dict[str, Any]) -> np.ndarray:
    """
    Elastic net logistic regression with fold-wise impute + scaling.
    """
    C = params.get("C", 0.2)
    l1_ratio = params.get("l1_ratio", 0.5)
    pipe = Pipeline(steps=[
        ("imp", SimpleImputer(strategy="median")),
        ("sc", StandardScaler(with_mean=True, with_std=True)),
        ("clf", LogisticRegression(

```

```

        penalty="elasticnet",
        solver="saga",
        C=C,
        l1_ratio=l1_ratio,
        max_iter=5000,
        n_jobs=-1,
        class_weight="balanced",
        random_state=params.get("seed", 42),
    ))
])
pipe.fit(X_tr.replace([np.inf, -np.inf], np.nan), y_tr)
return pipe.predict_proba(X_va.replace([np.inf, -np.inf], np.nan))[:, 1]

def fit_predict_tabpfn(X_tr: pd.DataFrame, y_tr: np.ndarray, X_va:
pd.DataFrame, params: Dict[str, Any]) -> np.ndarray:
    """
    Requires TabPFN installation. Must ensure feature count <= 500.
    TabPFN is explicitly positioned for small-to-medium datasets with <=500
    features.
    """
    try:
        from tabpfn import TabPFNClassifier
    except Exception as e:
        raise RuntimeError("TabPFN not installed: pip install tabpfn") from e

    max_features = params.get("max_features", 500)
    if X_tr.shape[1] > max_features:
        raise ValueError(f"TabPFN feature limit exceeded: {X_tr.shape[1]} >
{max_features}")

    # TabPFN expects numpy; it has its own preprocessing support.
    clf = TabPFNClassifier(device=params.get("device", "cuda" if
params.get("use_gpu", True) else "cpu"))
    clf.fit(X_tr.to_numpy(), y_tr)
    return clf.predict_proba(X_va.to_numpy())[:, 1]

```

Key citations for why these defaults are rational:

- Use LightGBM + Boruta/SHAP in proteomics with generalization emphasis (prot aging clock selected LightGBM for generalizability and used Boruta with SHAP shadow features, then SHAP-RFE). ¹⁵
- TabPFN's documented validated regime ($\leq 10k$ samples and ≤ 500 features) and minimal preprocessing. ⁴

Decision rules:

- If LightGBM FS1 baseline is already strong and stable (std AUC < 0.02), prioritize:
- feature subset refinement (FS1-SHAP top-k),

- seed averaging,
- ensemble gains (blend/stack), rather than heavy hyperparameter tuning.
- If XGBoost underperforms LightGBM by >0.01 consistently, keep it only if it adds ensemble diversity (check blend AUC).

Ensembling and stacking

OOF prediction and stacking are central competition mechanics; Kaggle’s guidance defines OOF predictions as cross-validated predictions for the training set and they are the correct basis for stacking and blending without leakage. ¹⁶

NVIDIA’s “Grandmaster Pro Tip” writeup is a modern example of how top solutions use stacking pipelines and OOF-driven ensembles to win tabular leaderboards. ¹⁷

[NEEDS-DATA] Seed averaging

- For each base model, run 3–5 seeds and average predicted probabilities.
- Store both OOF and test predictions per seed.

[NEEDS-DATA] OOF-tuned convex blending

- Start with a simple blend (e.g., 0.55 LGBM, 0.25 XGB, 0.20 Elastic).
- Tune weights by maximizing AUC on the OOF predictions.

[NEEDS-DATA] Ridge-regularized stacking

- Meta model: logistic regression with L2 penalty on OOF predictions (and optionally a few clinical covariates).
- Must not train meta model on in-fold predictions; only OOF.

[PRE-CODE] Blending and stacking utilities

```
# ensemble_utils.py
from __future__ import annotations

import numpy as np
from typing import Dict, List, Tuple
from sklearn.metrics import roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

def tune_convex_blend_weights(oof_preds: Dict[str, np.ndarray], y: np.ndarray,
                              step: float = 0.05) -> Tuple[Dict[str, float], float]:
    """
```

```

Grid search convex weights that sum to 1.0.
This is simple, robust, and fast for 3-4 models.
"""
keys = list(oof_preds.keys())
if len(keys) < 2:
    raise ValueError("Need >=2 models to blend.")

best_auc = -1.0
best_w = None

if len(keys) == 2:
    for w in np.arange(0, 1 + 1e-9, step):
        pred = w * oof_preds[keys[0]] + (1 - w) * oof_preds[keys[1]]
        auc = roc_auc_score(y, pred)
        if auc > best_auc:
            best_auc = auc
            best_w = {keys[0]: float(w), keys[1]: float(1 - w)}
    return best_w, float(best_auc)

if len(keys) == 3:
    for w0 in np.arange(0, 1 + 1e-9, step):
        for w1 in np.arange(0, 1 - w0 + 1e-9, step):
            w2 = 1 - w0 - w1
            pred = w0 * oof_preds[keys[0]] + w1 * oof_preds[keys[1]] + w2 *
oof_preds[keys[2]]
            auc = roc_auc_score(y, pred)
            if auc > best_auc:
                best_auc = auc
                best_w = {keys[0]: float(w0), keys[1]: float(w1), keys[2]:
float(w2)}
    return best_w, float(best_auc)

# For 4+ models, either use simplex sampling or a logistic meta-model.
raise NotImplementedError("Use stacking for 4+ models in this starter kit.")

def ridge_stack(oof_matrix: np.ndarray, y: np.ndarray, test_matrix: np.ndarray)
-> np.ndarray:
    """
    Ridge-stacked logistic regression.
    Inputs:
        oof_matrix: shape (n_samples, n_models)
        test_matrix: shape (n_test, n_models)
    """
    meta = Pipeline(steps=[
        ("sc", StandardScaler(with_mean=True, with_std=True)),
        ("clf", LogisticRegression(
            penalty="l2",
            solver="lbfgs",

```

```

        C=1.0,
        max_iter=5000,
        class_weight="balanced"
    ))
])
meta.fit(oof_matrix, y)
return meta.predict_proba(test_matrix)[: , 1]

```

Decision rules:

- If tuned blend improves OOF AUC by <0.001 over best single model, still keep it—small gains are often real on leaderboard.
- If stacking improves OOF AUC but **decreases** performance on a “proxy holdout” (e.g., one CV seed vs another), prefer the simpler blend.

Audit, ablation, and biological sanity checks

The BKBC ATI paper explicitly highlights top positive ATI proteins: **SPP1 (osteopontin)**, **MRC1**, **TNC**, with additional biologically relevant markers including **KIM-1/HAVCR1**, **WFDC2 (HE4)**, **GDF-15**, **SAA1**, **SAA2**, and the paper lists negative-associated proteins as well. ³

Ablations are not optional here because eGFR may act as a near-proxy; your model should show real incremental value from proteomics beyond clinical covariates if you want robustness. ³

[NEEDS-DATA] Required ablations

- Clinical-only vs Proteomics-only vs Combined:
- FS0 model (clinical)
- FS1-proteomics-only (156 proteins / 170 aptamers)
- FS1 combined (clinical + FS1)
- eGFR sensitivity:
- Run FS1 combined with eGFR removed.
- If AUC collapses, rebalance: prioritize proteomics signal and reduce the dominance of clinical proxies.

[NEEDS-DATA] SHAP top-10 plausibility check (final model)

- Compute SHAP top features on your final LightGBM model.
- Expect to see overlap with marker set (SPP1, MRC1, TNC, HAVCR1, etc.). If none appear, your model may be exploiting artifacts (e.g., batch covariates) or your mapping is broken.

[PRE-CODE] Ablation runner and sanity-check hooks

```

# audit_utils.py
from __future__ import annotations

```

```

import numpy as np
import pandas as pd
from typing import Dict, List, Optional, Callable, Any
from sklearn.metrics import roc_auc_score

def ablation_report(
    X: pd.DataFrame, y: np.ndarray,
    feature_sets: Dict[str, List[str]],
    fit_predict_fn: Callable[[pd.DataFrame, np.ndarray, pd.DataFrame, Dict[str,
Any]], np.ndarray],
    cv_runner, # expects run_oof_cv signature
    cv_cfg,
    fit_params: Dict[str, Any],
) -> pd.DataFrame:
    rows = []
    for name, cols in feature_sets.items():
        cols = [c for c in cols if c in X.columns]
        if len(cols) == 0:
            rows.append({"feature_set": name, "n_features": 0, "mean_auc":
np.nan, "std_auc": np.nan, "note": "no columns matched"})
            continue
        res = cv_runner(X[cols], y, fit_predict_fn=fit_predict_fn, cfg=cv_cfg,
groups=None, fit_params=fit_params)
        rows.append({"feature_set": name, "n_features": len(cols), "mean_auc":
res.mean_auc, "std_auc": res.std_auc, "note": ""})
    return pd.DataFrame(rows).sort_values("mean_auc", ascending=False)

def shap_top_features_lightgbm(X: pd.DataFrame, y: np.ndarray, lgb_params:
Dict[str, Any], top_n: int = 10) -> List[str]:
    """
    Train LGBM on full data (NOT for validation; for interpretability only at
the very end).
    """
    import lightgbm as lgb
    X = X.replace([np.inf, -np.inf], np.nan)
    model = lgb.LGBMClassifier(**{k:v for k,v in lgb_params.items() if k !=
"early_stopping_rounds"})
    model.fit(X, y)

    try:
        import shap
        expl = shap.TreeExplainer(model)
        X_sample = X.sample(n=min(2000, len(X)), random_state=0)
        shap_vals = expl.shap_values(X_sample)
        if isinstance(shap_vals, list):
            shap_vals = shap_vals[1]
        imp = np.mean(np.abs(shap_vals), axis=0)
        top_idx = np.argsort(-imp)[:top_n]

```

```

    return [X.columns[i] for i in top_idx]
except Exception:
    # fallback: gain importance
    imp = model.booster_.feature_importance(importance_type="gain")
    top_idx = np.argsort(-imp)[:top_n]
    return [model.booster_.feature_name()[i] for i in top_idx]

```

Decision rules:

- If combined model is only ≤ 0.005 AUC better than clinical-only, treat the leaderboard as likely dominated by clinical proxies; then:
- emphasize reliability (seed averaging + blending),
- emphasize “proteomics incremental signal” with carefully selected features (FS1-SHAP top-k).
- If proteomics-only model is competitive and stable, you have a strong chance to mitigate eGFR domain shift.

Prioritized hour-by-hour execution timeline

This is written to be used as a literal checklist during the 25-hour window.

Before the hackathon starts

[PRE-CODE] **Environment + repo**

- Prepare a clean repo with modules from this report.
- Preinstall / validate packages in a fresh environment:
- `pandas`, `numpy`, `scikit-learn`, `lightgbm`, `xgboost`, `shap`
- `tabPFN` (optional but recommended as an ensemble member once ≤ 500 features) ¹⁸
- Download and stash the BKBC Supplementary Data 1 spreadsheet locally (or embed lists as above). ⁹
- Pre-write a `run.py` orchestration script that:
- loads `train.csv/test.csv`,
- runs EDA,
- runs FS0/FS1 baselines,
- saves OOF and test preds.

Hour-by-hour hackathon plan

[NEEDS-DATA] **Hour 0–1: schema + sanity**

- Load data; run `basic_eda()`; print summary.
- Confirm target type and unique values.
- Identify ID column if present; check duplicates/near duplicates.

Decision to move on:

- Move on once you can run a 5-fold CV for a toy baseline without crashes and you have a clear `X`, `y`, feature columns definition.

[NEEDS-DATA] Hour 1–3: FS0 and FS1 LightGBM baseline

- Build FS0 (clinical-only) columns.
- Build FS1 columns using `select_fs1_columns()`.
- Run LightGBM OOF on FS0 and FS1.
- Record mean/std AUC.

Decision to move on:

- Move on once FS1 is matching a plausible number of features and CV variance is not pathological.

[NEEDS-DATA] Hour 3–6: add XGBoost + elastic net; establish baseline ensemble

- Run XGBoost on FS1 (same folds).
- Run elastic net on:
 - FS0 (clinical-only)
 - a reduced proteomics set (FS1 only, perhaps top-k by filter first to keep linear model feasible).
- Generate OOF preds; tune a convex blend of 2–3 models. ¹⁶

Decision to move on:

- If blend improves AUC meaningfully (>0.002) and is stable, proceed to feature selection refinement.

[NEEDS-DATA] Hour 6–11: FS2 creation inside CV

- Within each fold:
 - candidate pool = FS1 + top-k MI from remaining proteins (k 500–1500).
 - train LightGBM; rank by SHAP/gain; keep top-k (try k 200/400/800).
 - Compare FS1 vs FS1-SHAP(k) vs FS2-SHAP(k).

Decision to move on:

- Stop adding features when AUC plateaus or fold variance rises.

[NEEDS-DATA] Hour 11–15: stability wrapper and seed averaging

- Run 3 seeds for LightGBM on best feature set.
- Run 2–3 seeds for XGBoost.
- Average predictions (per-model seed average), then retune blend weights.

Decision to move on:

- If seed averaging reduces std AUC and improves mean AUC even slightly, lock it in.

[NEEDS-DATA] Hour 15–19: optional Boruta-style selection on reduced pool

- Only if you have time and candidate pool ≤ 1500 :
- `boruta_shadow_select()` inside fold.
- Compare to SHAP top-k approach.

Decision to move on:

- If Boruta set is unstable across folds or too slow, abandon and keep SHAP top-k.

[NEEDS-DATA] Hour 19–22: TabPFN as a reduced-feature ensemble member

- Build ≤ 500 feature matrix:
- clinical + FS1 + (if needed) add top SHAP features until ≤ 500 .
- Run TabPFN OOF; add to blend. TabPFN is designed for minimal preprocessing and excels in its validated regime. ⁴
- If TabPFN is not available, skip without regret.

Decision to move on:

- Keep TabPFN only if it adds ensemble lift or improves stability.

[NEEDS-DATA] Hour 22–24: audit and ablations

- Run clinical-only vs proteomics-only vs combined.
- Remove eGFR and re-run combined model.
- Compute final SHAP top-10 and check overlap with known ATI markers highlighted in the paper (SPP1, MRC1, TNC, HAVCR1, etc.). ³

Decision to move on:

- If eGFR removal collapses AUC drastically, consider submitting an ensemble that includes a “proteomics-heavy” model to hedge generalization.

[NEEDS-DATA] Hour 24–25: finalize and submit

- Train final models on full training set (per model, with selected features).
- Generate test predictions.
- Save artifacts:
- selected feature list(s),
- OOF predictions,
- blend weights,
- SHAP top-features list for writeup.

Implementation wiring example

This is the minimal glue you’ll run once you have data.

[PRE-CODE] Orchestration skeleton

```
# run_pipeline.py
from __future__ import annotations

import numpy as np
import pandas as pd

from eda_utils import basic_eda, print_eda_summary, guess_schema
from cv_harness import CVConfig, run_oof_cv
from bkbc_fs1 import select_fs1_columns
from models_and_fitfns import (
    make_lgb_params, make_xgb_params,
    fit_predict_lightgbm, fit_predict_xgboost, fit_predict_elasticnet
)
from ensemble_utils import tune_convex_blend_weights
from audit_utils import ablation_report

def main(train_path: str, test_path: str, target_col: str, id_col: str | None =
None):
    train = pd.read_csv(train_path)
    test = pd.read_csv(test_path)

    eda = basic_eda(train, target_col=target_col, id_col=id_col)
    print_eda_summary(eda)

    schema = guess_schema(train, target_col=target_col, id_col=id_col)
    y = train[target_col].to_numpy()

    # Define clinical columns (NEEDS-DATA: refine based on real schema)
    clinical_cols = schema.clinical_cols
    if len(clinical_cols) == 0:
        # fallback: treat non-protein numeric columns as clinical if SeqId
columns exist
        clinical_cols = [c for c in train.columns if c not in
schema.protein_cols and c not in [target_col] + schema.id_cols]

    # FS0 and FS1
    fs0_cols = [c for c in clinical_cols if c in train.columns]
    fs1_cols, fs1_diag = select_fs1_columns(train.drop(columns=[target_col]),
clinical_cols=fs0_cols)
    print("FS1 selection diagnostics:", fs1_diag)

    # Build feature matrices
    X0 = train[fs0_cols].copy()
    X1 = train[fs1_cols].copy()
```

```

    cv_cfg = CVConfig(n_splits=5, random_state=42, group_col=(id_col if id_col
in train.columns else None))
    groups = train[id_col].to_numpy() if (id_col and id_col in train.columns
and cv_cfg.group_col) else None

    # LightGBM baseline on FS1
    lgb_params = make_lgb_params(seed=42)
    res_lgb = run_oof_cv(X1, y, fit_predict_fn=fit_predict_lightgbm,
cfg=cv_cfg, groups=groups, fit_params=lgb_params)

    # XGBoost baseline on FS1
    xgb_params = make_xgb_params(seed=42)
    res_xgb = run_oof_cv(X1, y, fit_predict_fn=fit_predict_xgboost, cfg=cv_cfg,
groups=groups, fit_params=xgb_params)

    # Elastic net on FS0 (clinical only)
    en_params = {"C": 0.2, "l1_ratio": 0.5, "seed": 42}
    res_en0 = run_oof_cv(X0, y, fit_predict_fn=fit_predict_elasticnet,
cfg=cv_cfg, groups=groups, fit_params=en_params)

    # Blend tuning
    oof_preds = {"lgb": res_lgb.oof_pred, "xgb": res_xgb.oof_pred, "en0":
res_en0.oof_pred}
    w, auc = tune_convex_blend_weights(oof_preds, y, step=0.05)
    print("Best blend weights:", w, "AUC:", auc)

    # Ablations quick report
    feats = {"FS0_clinical": fs0_cols, "FS1_combined": fs1_cols}
    rep = ablation_report(train.drop(columns=[target_col]), y, feats,
fit_predict_lightgbm, run_oof_cv, cv_cfg, lgb_params)
    print(rep)

if __name__ == "__main__":
    # Example:
    # main("train.csv", "test.csv", target_col="ati_label", id_col="patient_id")
    pass

```

Why this plan works for your specific hackathon goal

- You exploit the strongest available prior: the BKBC ATI Supplementary Data 1 biomarker list (156 proteins; 170 aptamers). ⁹
- You borrow a proven, implementable proteomics feature selection workflow (Boruta + SHAP shadow features + SHAP-RFE) from a large proteomics modeling paper that selected LightGBM specifically for cross-cohort generalizability. ¹⁹
- You protect yourself against self-inflicted leaderboard hallucinations by enforcing fold-wise preprocessing/selection—directly aligned with scikit-learn’s warnings and empirical bias demonstrations. ⁷

- You deploy TabPFN and other TFM only where they are validated: after reducing to ≤ 500 features, consistent with the TabPFN paper’s explicit “when to use” guidance. 4

1 5 6 https://somalogic.com/wp-content/uploads/2022/07/SL00000048_Rev-3_2022-01_-Data-Standardization-and-File-Specification-Technical-Note-v2.pdf

https://somalogic.com/wp-content/uploads/2022/07/SL00000048_Rev-3_2022-01_-Data-Standardization-and-File-Specification-Technical-Note-v2.pdf

2 3 8 Plasma proteomics of acute tubular injury | Nature Communications

<https://www.nature.com/articles/s41467-024-51304-x>

4 13 Accurate predictions on small data with a tabular foundation model | Nature

<https://www.nature.com/articles/s41586-024-08328-6>

7 https://scikit-learn.org/stable/common_pitfalls.html

https://scikit-learn.org/stable/common_pitfalls.html

9 NCOMMS-24-07572B doasf

https://static-content.springer.com/esm/art%3A10.1038%2Fs41467-024-51304-x/MediaObjects/41467_2024_51304_MOESM3_ESM.pdf

10 15 19 <https://www.nature.com/articles/s41591-024-03164-7>

<https://www.nature.com/articles/s41591-024-03164-7>

11 Discovery of sparse, reliable omic biomarkers with Stabl | Nature Biotechnology

<https://www.nature.com/articles/s41587-023-02033-x>

12 https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html

https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html

14 [2502.05564] TabICL: A Tabular Foundation Model for In-Context Learning on Large Data

<https://arxiv.org/abs/2502.05564>

16 <https://www.kaggle.com/getting-started/177975>

<https://www.kaggle.com/getting-started/177975>

17 Grandmaster Pro Tip: Winning First Place in a Kaggle ...

https://developer.nvidia.com/blog/grandmaster-pro-tip-winning-first-place-in-a-kaggle-competition-with-stacking-using-cuml/?utm_source=chatgpt.com

18 PriorLabs/TabPFN - Foundation Model for Tabular Data

https://github.com/PriorLabs/TabPFN?utm_source=chatgpt.com